# DSA 8070 R Session 1: Exploratory Analysis of Multivariate Data

Whitney Huang, Clemson University

## Contents



## Overview

This first R session introduces several basic tools for exploring multivariate data.

The emphasis is on two questions:

1. **How do we numerically summarize variation and dependence among multiple variables?**
2. **How can we visualize multivariate structure that cannot be seen from one variable at a time?**

We will begin with covariance and generalized variance, and then explore several graphical tools for multivariate data.

Throughout the session, pay attention not only to the R syntax, but also to the statistical meaning of the numerical summaries and plots.

## Descriptive Statistics

### Sample Covariance Visualization

Covariance measures the extent to which two variables vary together.

To illustrate this idea, we simulate a bivariate normal sample

$$\mathbf{X} = \begin{pmatrix} X_j \\ X_k \end{pmatrix} \sim N_2 \left[ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 4 & 1.4 \\ 1.4 & 1 \end{pmatrix} \right].$$

Thus,

- $\mathrm{Var}(X_j) = 4$,
- $\mathrm{Var}(X_k) = 1$, and
- $\mathrm{Cov}(X_j, X_k) = 1.4$.

The corresponding population correlation is

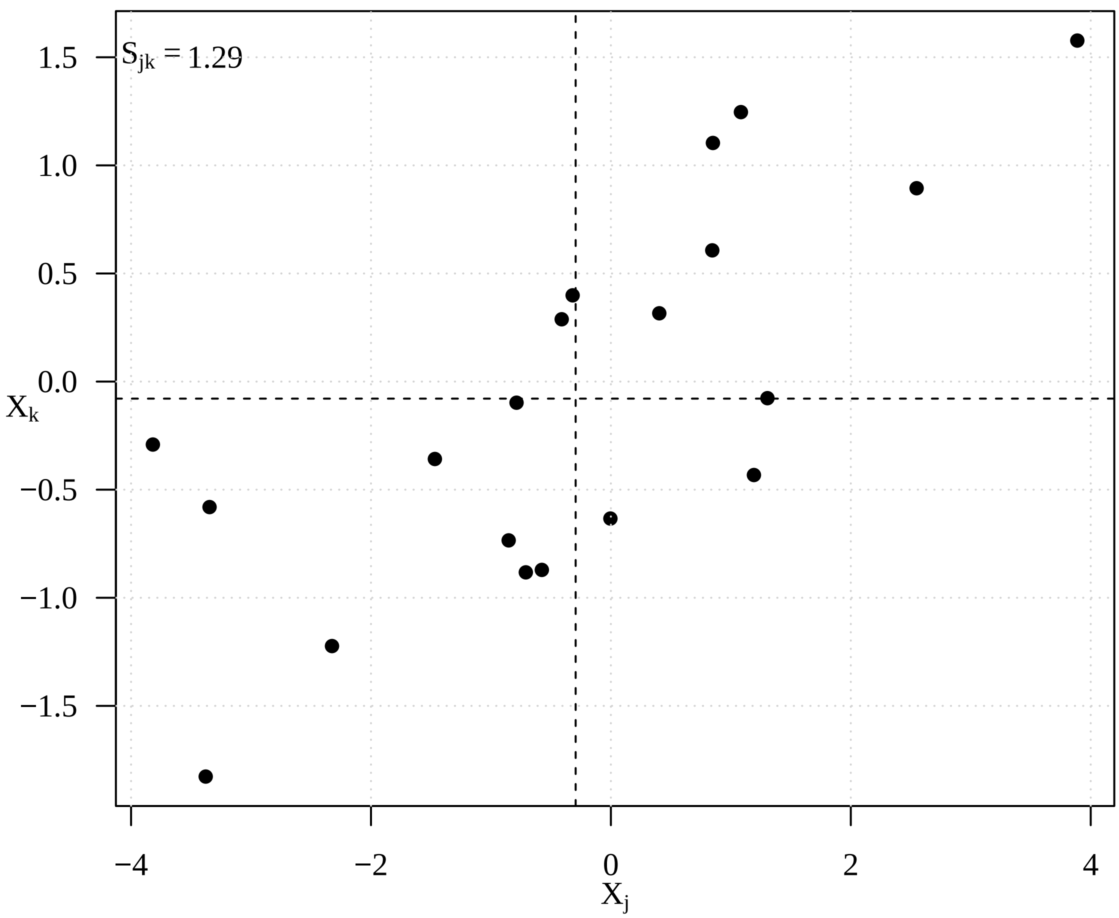
$$\rho_{jk} = \frac{1.4}{\sqrt{4}\sqrt{1}} = 0.7.$$

Because the covariance is positive, observations for which $X_j$ is above its mean tend also to have $X_k$ above its mean, and observations below one mean tend to be below the other.

We first visualize this relationship using a scatterplot.

```r
set.seed(123)

# MASS contains mvrnorm(), which generates data from a
# multivariate normal distribution.
library(MASS)

# Population covariance matrix:
# Var(X_j) = 4, Var(X_k) = 1, Cov(X_j, X_k) = 1.4.
Sigma <- matrix(c(4, 1.4, 1.4, 1), nrow = 2)

# Generate n = 20 observations from the bivariate normal distribution.
dat <- mvrnorm(n = 20, mu = c(0, 0), Sigma = Sigma)

# Number of observations in the simulated sample.
n <- nrow(dat)

# Calculate the sample covariance.
sample_cov <- cov(dat[, 1], dat[, 2])

par(mar = c(3.5, 3.5, 0.8, 0.5), mgp = c(2, 1, 0), las = 1, family = "serif")

plot(dat, pch = 16, xlab = "", ylab = "")

# Add mathematical axis labels.
mtext(expression(X[j]), side = 1, line = 2)
mtext(expression(X[k]), side = 2, line = 2)
```

```r
# Display the sample covariance on the plot.
text(-3.8, 1.5, expression(paste(S[jk], " = ")))
text(-3.3, 1.5, round(sample_cov, 2))

# Dashed lines indicate the sample means.
abline(h = mean(dat[, 2]), lty = 2)
abline(v = mean(dat[, 1]), lty = 2)
grid()
```




The dashed lines divide the scatterplot according to whether each variable is above or below its sample mean.

For positive covariance, we expect relatively many observations in the upper-right and lower-left quadrants. These observations contribute positively to

$$(X_{ij} - \bar{X}_j)(X_{ik} - \bar{X}_k).$$

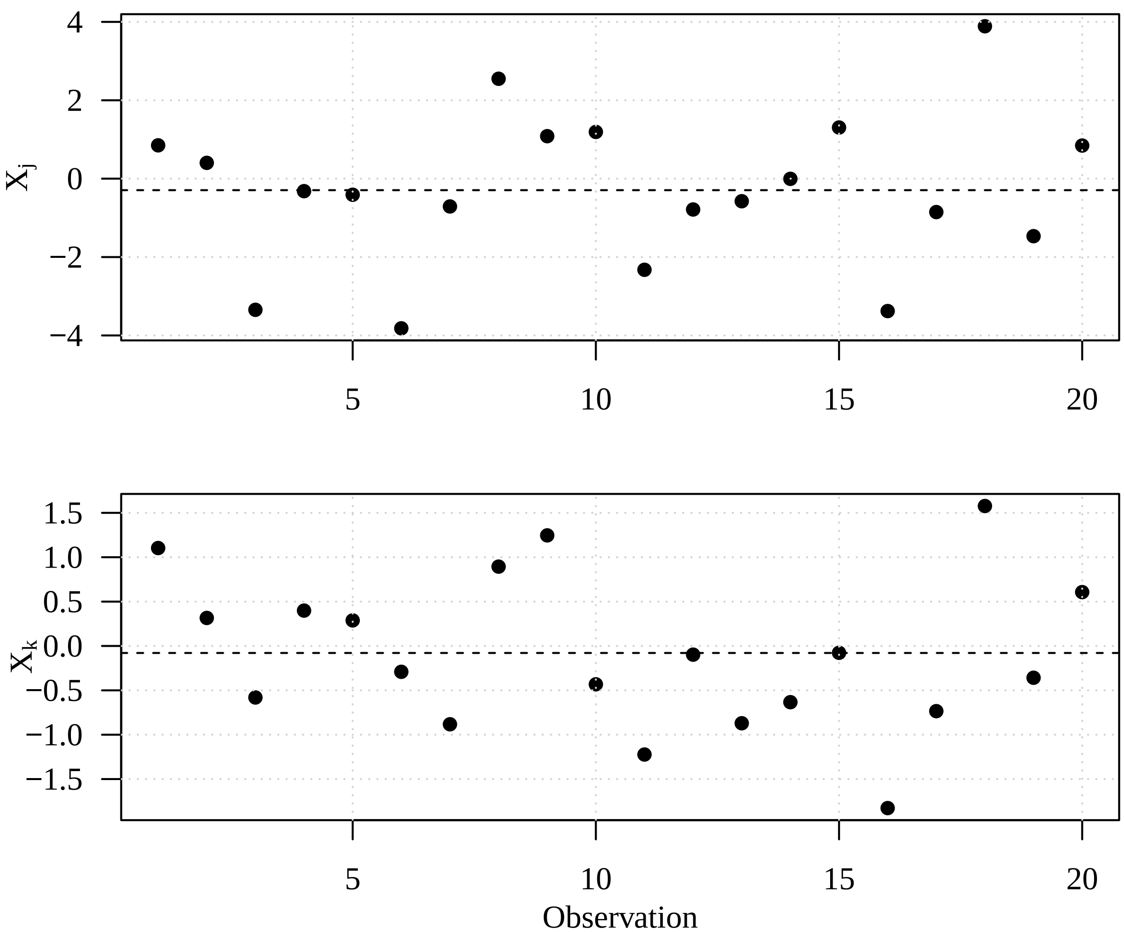
Observations in the upper-left or lower-right quadrants contribute negatively to the covariance.

### Viewing Covariance through Co-movement

Another way to understand covariance is to examine the two variables across the same sequence of observations.

The following two run plots use the same horizontal index $i = 1, \ldots, n$. If the two variables have positive covariance, observations that are above their respective means in one plot will often also be above the mean in the other plot.

```
par(mfrow = c(2, 1), mar = c(3.5, 3.5, 0.5, 0.5), mgp = c(2, 1, 0), las = 1,
    family = "serif")

# First variable
plot(1:n, dat[, 1], pch = 16, xlab = "", ylab = expression(X[j]))
grid()
abline(h = mean(dat[, 1]), lty = 2)

# Second variable
plot(1:n, dat[, 2], pch = 16, xlab = "Observation", ylab = expression(X[k]))
grid()
abline(h = mean(dat[, 2]), lty = 2)
```




```
# Return to a single plotting panel for later figures.
par(mfrow = c(1, 1))
```

Compare the two panels vertically. When $X_j$ is unusually large or small, does $X_k$ tend to move in the same direction?

This provides another intuitive interpretation of **positive covariance as co-movement**.

### Sample Covariance versus Population Covariance

The covariance matrix is a population quantity, but in practice it is usually unknown and must be estimated from a sample.

Even if the true population covariance is zero, the sample covariance will almost never be exactly zero because of sampling variability.

To illustrate this, suppose

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \sim N_2 \left[ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right].$$

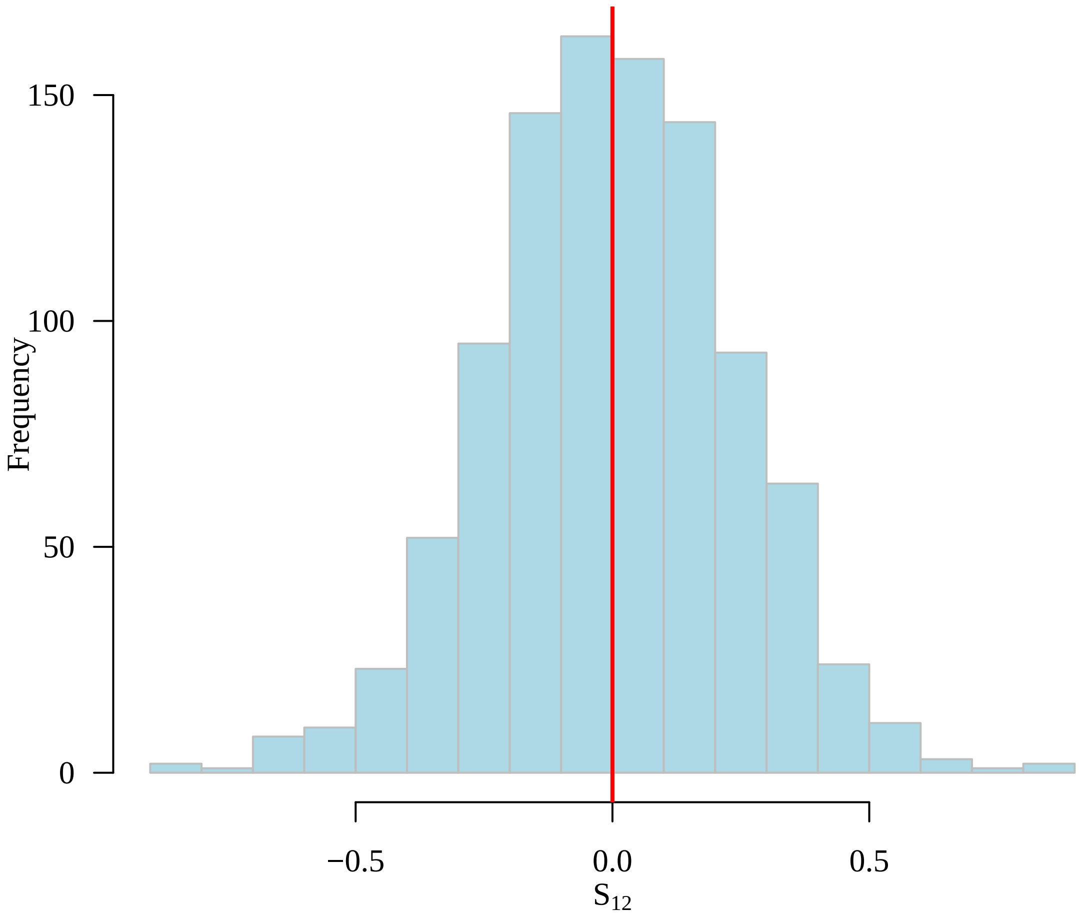
The population covariance is therefore

$$\sigma_{12} = 0.$$

We repeatedly generate samples of size $n = 20$, calculate the sample covariance $s_{12}$, and examine its sampling distribution.

This is an example of a **Monte Carlo experiment**: repeated random simulation is used to study the behavior of a statistic.

```r
set.seed(123)

# Generate 1,000 independent samples.
#
# Each call to mvrnorm() produces a 20 x 2 matrix.
# replicate() stores these matrices in a 20 x 2 x 1000 array.
sim_dat <- replicate(1000, mvrnorm(n = 20, mu = c(0, 0),
                                   Sigma = matrix(c(1, 0, 0, 1), nrow = 2)
                                   )
                     )

# For each simulated data set, calculate the sample covariance
# between the two variables.
s12 <- apply(sim_dat, 3, function(x) cov(x[, 1], x[, 2]))

# Examine the empirical sampling distribution of s_12.
par(mar = c(3.5, 3.5, 0.8, 0.5), mgp = c(2, 1, 0), las = 1, family = "serif")
hist(s12, breaks = 20, col = "lightblue", border = "gray", main = "",
     xlab = expression(S[12]))

# The true population covariance is zero.
abline(v = 0, col = "red", lwd = 2)
```




```r
# Monte Carlo average of the sample covariance.
mean(s12)
```

```
## [1] 0.000108218
```

```r
# Monte Carlo standard deviation of the sample covariance.
sd(s12)
```

```
## [1] 0.2397229
```

Several points are worth noticing:

1. Individual sample covariances can be noticeably different from zero even though the true covariance is zero.
2. The sampling distribution is centered approximately at the population value, 0.
3. The spread of this distribution reflects sampling uncertainty.
4. With a larger sample size, we would expect the distribution of $s_{12}$ to become more concentrated around zero.

### A Small Bivariate Example

For a small dataset, we can calculate the mean vector, covariance matrix, and correlation matrix directly.

Each row below represents one observation on two variables, $X_1$ and $X_2$.

```r
# Create a 5 x 2 data matrix.
small_dat <- cbind(x1 = c(42, 52, 88, 58, 60),
                   x2 = c(4, 5, 7, 4, 5))

small_dat
```

```
##      x1 x2
## [1,] 42  4
## [2,] 52  5
## [3,] 88  7
## [4,] 58  4
## [5,] 60  5
```

```r
# Sample mean vector.
(means <- colMeans(small_dat))
```

```
## x1 x2
## 60  5
```

```r
# Sample covariance matrix.
(S <- cov(small_dat))
```

```
##     x1   x2
## x1 294 19.0
## x2  19  1.5
```

```r
# Sample correlation matrix.
(R <- cor(small_dat))
```

```
##           x1        x2
## x1 1.0000000 0.9047619
## x2 0.9047619 1.0000000
```

For a covariance matrix

$$S = \begin{pmatrix} s_{11} & s_{12} \\ s_{21} & s_{22} \end{pmatrix},$$

the diagonal entries $s_{11}$ and $s_{22}$ are the sample variances, while the off-diagonal entry $s_{12} = s_{21}$ is the sample covariance.

The correlation matrix standardizes each covariance:

$$r_{jk} = \frac{s_{jk}}{\sqrt{s_{jj} s_{kk}}}.$$

Consequently, correlations are dimensionless and always lie between $-1$ and 1.

### Generliazed Variance

For a single variable, the variance measures spread.

For $p$ variables, the covariance matrix $S$ contains information about spread in all directions. One scalar summary of this multivariate spread is the **generalized variance**:

$$|S| = \det(S).$$

Geometrically, the determinant is related to the volume of the multivariate covariance ellipsoid.

A larger determinant generally corresponds to greater overall multivariate dispersion, although its magnitude also depends strongly on the units and scales of the variables.

```r
data(mtcars)

# Select five quantitative variables from mtcars.
vars <- c("mpg", "disp", "hp", "drat", "wt")

car <- mtcars[, vars]

# Sample covariance matrix.
S <- cov(car)

S
```

```
##              mpg        disp         hp        drat          wt
## mpg    36.324103   -633.09721 -320.73206   2.1950635  -5.1166847
## disp -633.097208  15360.79983 6721.15867 -47.0640192 107.6842040
## hp   -320.732056   6721.15867 4700.86694 -16.4511089  44.1926613
## drat    2.195064    -47.06402  -16.45111   0.2858814  -0.3727207
## wt     -5.116685    107.68420   44.19266  -0.3727207   0.9573790
```

```r
# Generalized variance.
(genVar <- det(S))
```

```
## [1] 3951786
```

For the bivariate covariance matrix

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix},$$

the determinant is

$$|\Sigma| = \sigma_1^2\sigma_2^2(1-\rho^2).$$

Therefore, if the marginal variances are held fixed, stronger correlation in either direction reduces the generalized variance.

If the variables are uncorrelated, $\rho = 0$, then

$$|\Sigma| = \sigma_1^2\sigma_2^2,$$

which is simply the product of the marginal variances.

```r
set.seed(123)

# Correlated population
dat_cor <- mvrnorm(n = 100, mu = c(0, 0),
                   Sigma = matrix(c(4, 1.4, 1.4, 1), nrow = 2)
)

det(cov(dat_cor))
```

```
## [1] 1.585516
```

```r
set.seed(123)

# Uncorrelated population with the same marginal variances
dat_uncor <- mvrnorm(n = 100, mu = c(0, 0),
                     Sigma = matrix(c(4, 0, 0, 1), nrow = 2)
)

det(cov(dat_uncor))
```

```
## [1] 3.108855
```

**Sample versus Population Generalized Variance** The generalized variance can refer either to

$$|\Sigma|,$$

the population generalized variance, or

$$|S|,$$

the sample generalized variance.

Because $S$ is estimated from a finite sample, $|S|$ varies from sample to sample. As the sample size increases, we generally expect $S$ to approach $\Sigma$, and therefore $|S|$ to approach $|\Sigma|$.

```r
# Correlated case
det(cov(dat_cor))   # sample generalized variance
```

```
## [1] 1.585516
```

```r
det(matrix(c(4, 1.4, 1.4, 1), nrow = 2))
```

```
## [1] 2.04
```

```r
# population generalized variance


# Uncorrelated case
det(cov(dat_uncor)) # sample generalized variance
```

```
## [1] 3.108855
```

```r
det(matrix(c(4, 0, 0, 1), nrow = 2))
```

```
## [1] 4
```

```r
# population generalized variance
```

## Graphs and Visualization

Numerical summaries such as the mean vector, covariance matrix, and correlation matrix provide compact descriptions of multivariate data.

However, numerical summaries alone can hide important features such as

- nonlinear relationships,
- clusters,
- group separation,
- unusual observations, and
- interactions among several variables.

We therefore complement numerical summaries with graphical displays. There is no single perfect visualization for high-dimensional data; each graph emphasizes a different aspect of multivariate structure.

### Scatterplot Matrix: `pairs()`

When a dataset contains several quantitative variables, it is often useful to look at all pairwise relationships simultaneously.

A **scatterplot matrix** displays a scatterplot for every pair of variables. For $p$ quantitative variables, the matrix contains $p \times p$ panels.

This allows us to quickly examine:

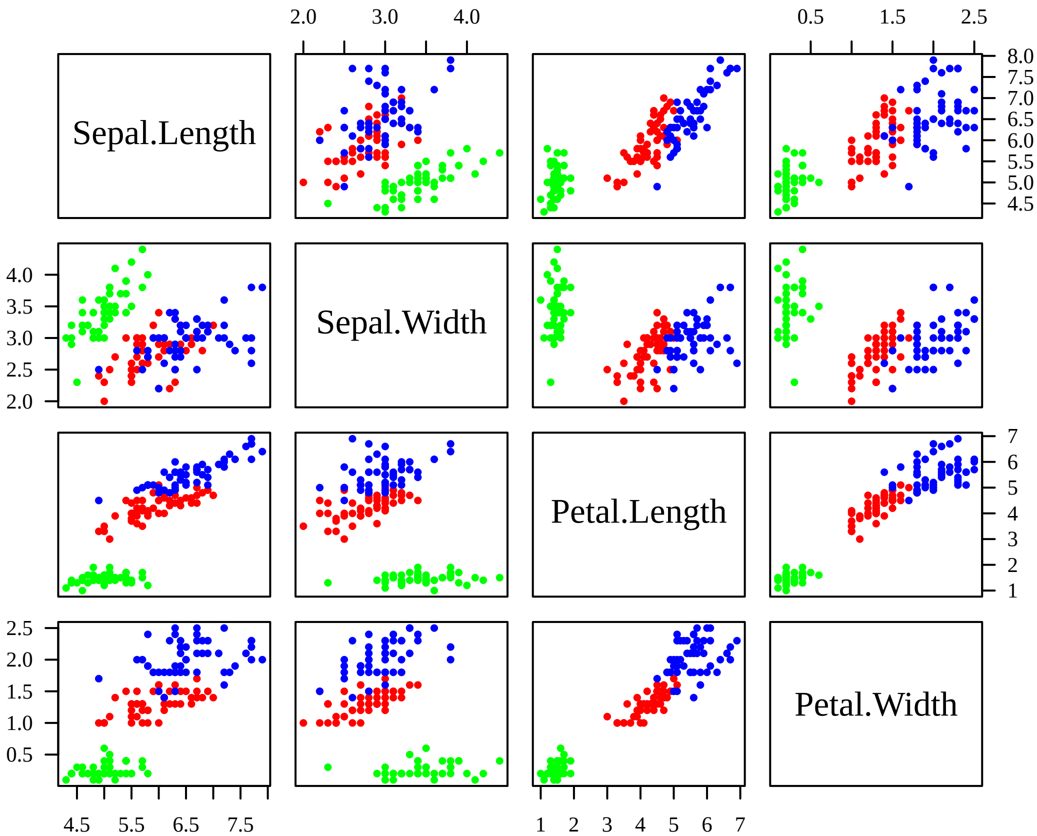
- the direction and strength of pairwise associations;
- possible nonlinear relationships;
- clusters or group separation;
- unusual observations; and
- variables that may provide similar information.

For the `iris` data, we use color to indicate species. This allows us to investigate not only associations among the four measurements, but also whether different species occupy distinct regions of the multivariate measurement space.

```r
# Display the first few observations.
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

```r
# The first four columns are quantitative measurements.
# The fifth column, Species, is categorical and is therefore
# excluded from the scatterplot matrix.

# Create one plotting color for each species.
# iris contains 50 observations from each of the three species.
species_col <- rep(c("green", "red", "blue"), each = 50)

par(mar = c(3.5, 3.5, 0.8, 0.5), mgp = c(2, 1, 0), las = 1, family = "serif")
pairs(iris[, -5], col = species_col, pch = 16, cex = 0.8)
```




Look across the panels rather than interpreting each scatterplot in isolation.

In particular, notice that `Petal.Length` and `Petal.Width` have a very strong positive relationship. The petal measurements also provide much clearer separation among species than the sepal measurements.

This illustrates an important multivariate idea: **different variables may contain very different amounts of information about group structure.**

### Enhanced Scatterplot Matrix: `ggpairs()`

The base R function `pairs()` provides a compact view of pairwise relationships. The `ggpairs()` function from the **GGally** package extends this idea by combining several types of information in one display.

Depending on the data and settings, a `ggpairs()` display may include:

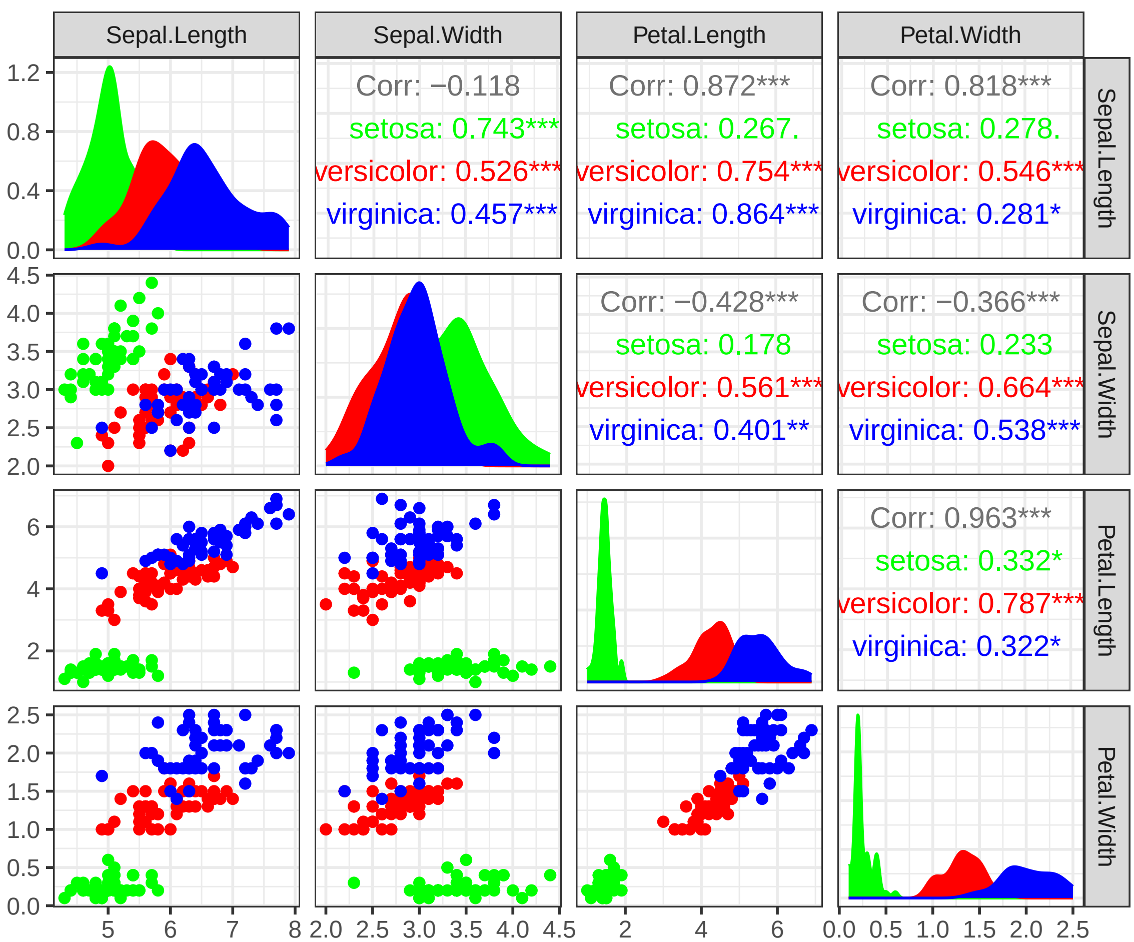
- scatterplots for pairs of quantitative variables;

- marginal density plots or histograms;
- correlation coefficients; and
- group-specific patterns.

Thus, `ggpairs()` provides both **univariate** and **bivariate** information within a single multivariate display.

Here we again use species as a grouping variable.

```
library(GGally)
library(ggplot2)

ggpairs(iris, columns = 1:4,
        mapping = aes(color = Species, fill = Species)
) +
  theme_bw() +
  scale_color_manual(
    values = c("green", "red", "blue")
  ) +
  scale_fill_manual(
    values = c("green", "red", "blue")
  )
```




Compared with `pairs()`, this figure contains considerably more information.

For example, we can simultaneously examine:

- the marginal distribution of each measurement;
- pairwise associations among measurements;
- correlations between variables; and
- differences among the three species.

### Three-Dimensional Scatterplot

A standard scatterplot displays two quantitative variables at a time. A three-dimensional scatterplot allows us to display three quantitative variables simultaneously.

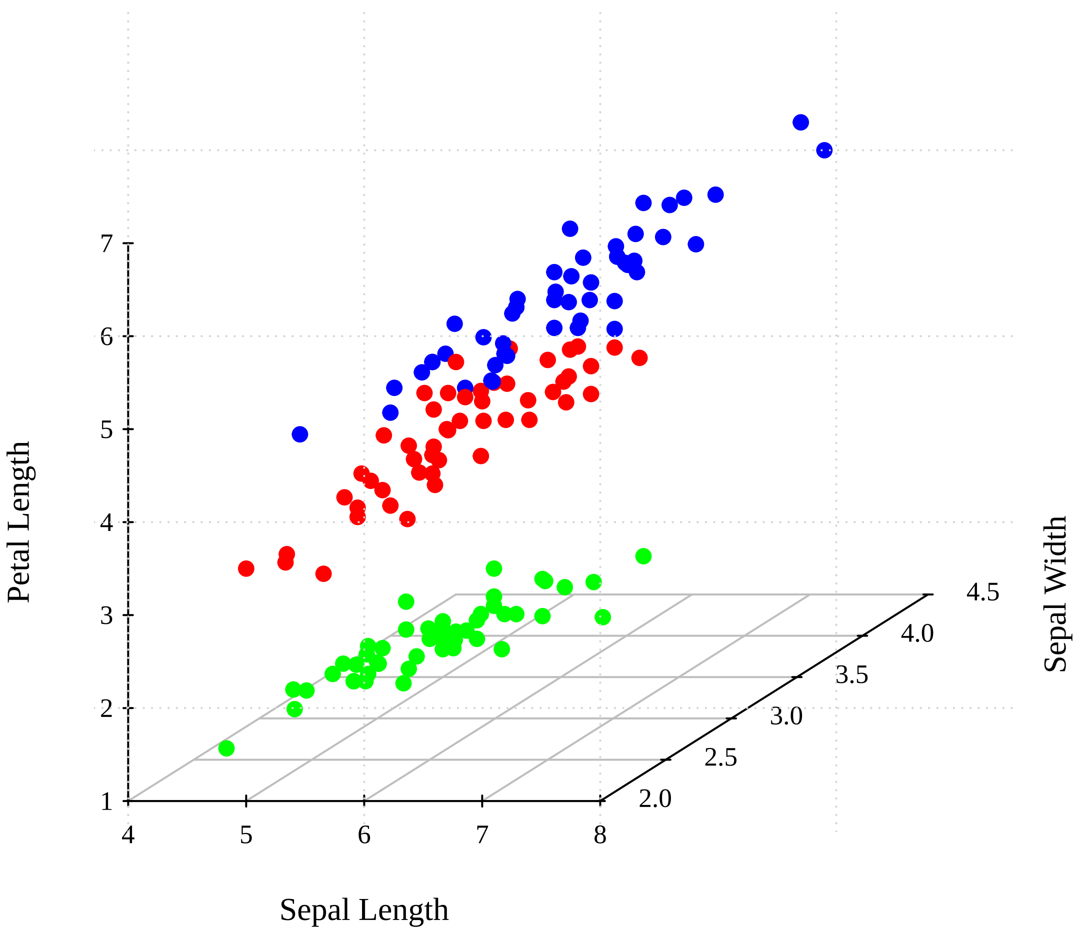
Here we use the first three iris measurements:

$$(\text{Sepal Length},\ \text{Sepal Width},\ \text{Petal Length}).$$

Each flower is represented by one point in three-dimensional space, and color indicates species.

A 3D plot can reveal structure that is difficult to see in a single 2D scatterplot. However, interpretation depends on the viewing angle, and overlapping points may make the display difficult to read. Therefore, 3D visualization should usually complement rather than replace simpler pairwise plots.

```r
library(scatterplot3d)

species_col <- rep(c("green", "red", "blue"), each = 50)

par(family = "serif", las = 1)
scatterplot3d(iris[, 1:3], pch = 19,
  color = species_col, grid = TRUE,
  box = FALSE, mar = c(3, 3, 0.5, 3),
  xlab = "Sepal Length", ylab = "Sepal Width", zlab = "Petal Length")
grid()
```




Ask yourself whether the three species appear more clearly separated when three measurements are considered simultaneously than when only two are used.

Also notice a limitation: the apparent amount of separation can depend on the orientation of the plot. This is one reason that statistical dimension reduction methods, introduced later in the course, are often preferable to simply plotting arbitrary triples of variables.

### Parallel Coordinate Plot

Scatterplots focus primarily on relationships between pairs or triples of variables. A **parallel coordinate plot** provides a different way to visualize all variables simultaneously.

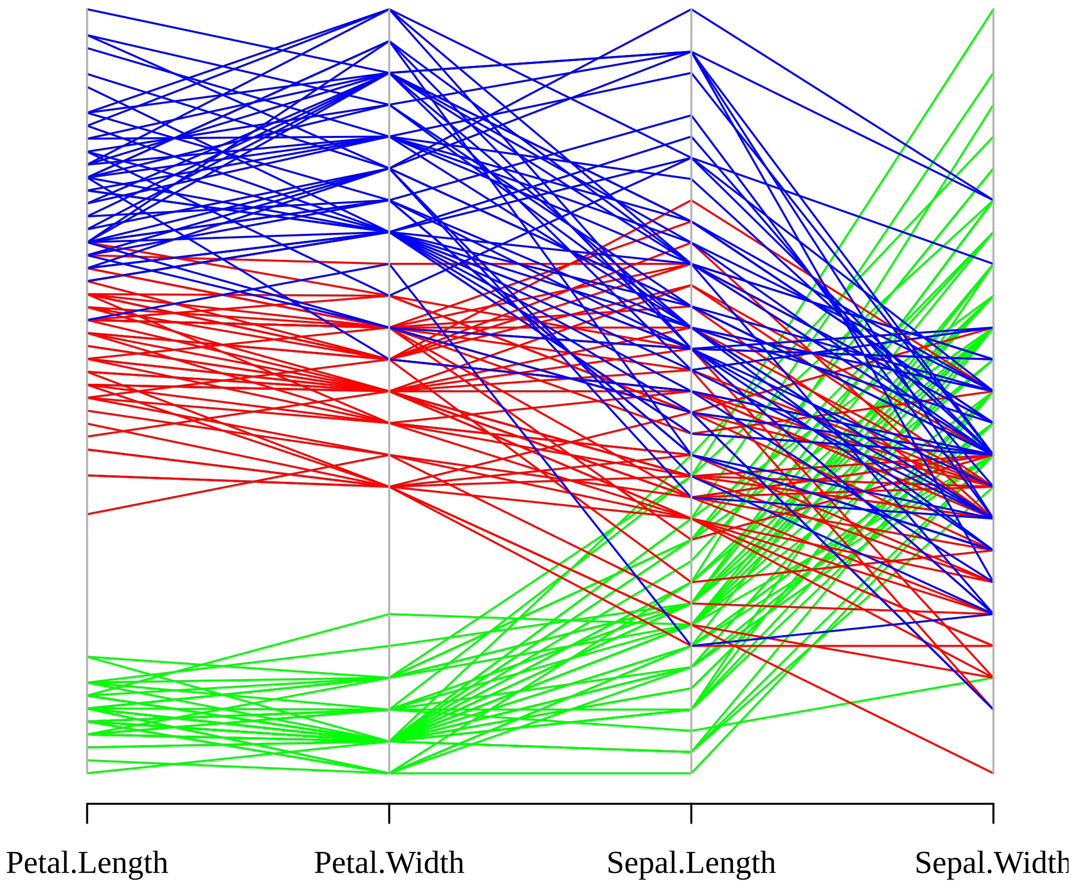
Each quantitative variable is represented by a vertical axis. One observation is represented by a line connecting its values across the axes.

Thus,

- each **axis** represents a variable;
- each **line** represents one observational unit; and
- similar line patterns indicate observations with similar multivariate profiles.

Parallel coordinates are especially useful for identifying groups, multivariate patterns, and unusual observations.

```r
# Keep the four quantitative iris measurements.
dat <- iris[, 1:4]

par(mar = c(3, 3.5, 0.5, 1), mgp = c(2, 1, 0), las = 1, family = "serif")

parcoord(dat[, c(3, 4, 1, 2)],  # Change the order of the displayed variables
         col = species_col)
```




The variables are deliberately reordered so that the two petal measurements appear first. Axis ordering matters in a parallel coordinate plot because adjacent axes are easier to compare visually.

#### Chernoff Faces

Chernoff faces provide an unusual approach to multivariate visualization.

Different quantitative variables are mapped to different facial features, such as:

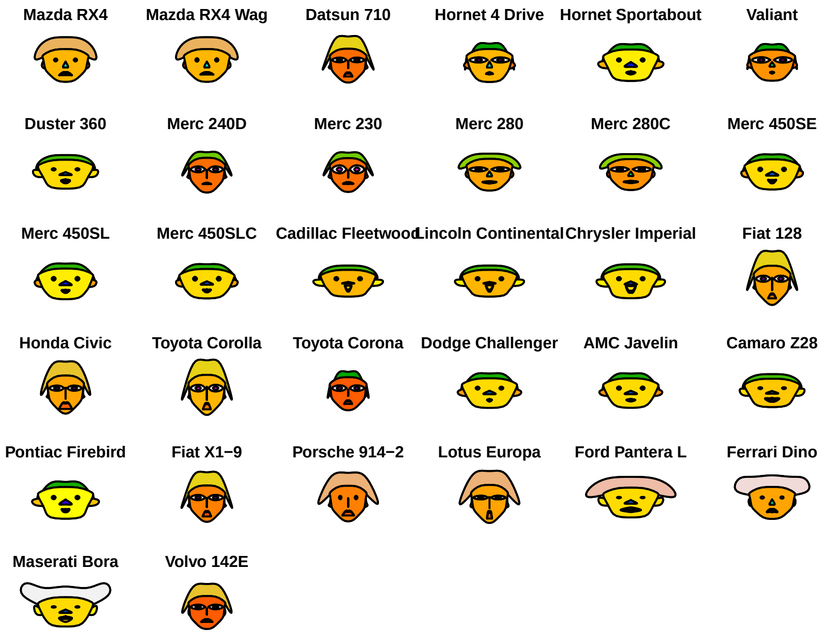
- face shape,
- eye size,
- mouth curvature, and
- nose size.

Each observation is therefore represented by a face.

The idea is that humans are particularly sensitive to differences among faces, so multivariate observations with similar values may produce visually similar facial patterns.

Chernoff faces are useful for illustrating the challenge of representing many variables simultaneously, although they are used much less frequently in modern statistical practice than scatterplot matrices, heatmaps, or dimension-reduction methods.

```r
library(aplpack)

# Remove outer margins so the faces use the available plotting space.
par(mar = rep(0, 4))

# Each row of mtcars becomes one face.
faces(mtcars, cex = 0.8)
```

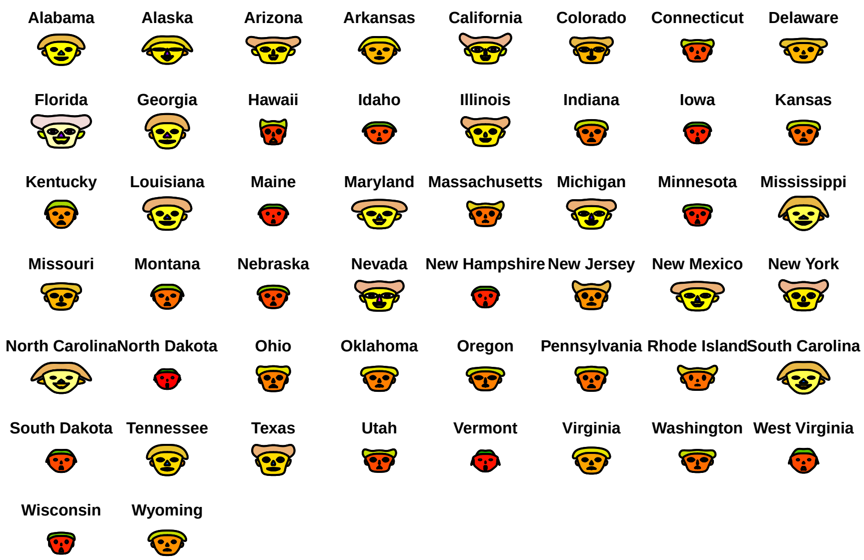



```
## effect of variables:
##  modified item       Var
##  "height of face   " "mpg"
##  "width of face    " "cyl"
##  "structure of face" "disp"
##  "height of mouth  " "hp"
##  "width of mouth   " "drat"
##  "smiling          " "wt"
##  "height of eyes   " "qsec"
##  "width of eyes    " "vs"
##  "height of hair   " "am"
##  "width of hair   "  "gear"
##  "style of hair   "  "carb"
##  "height of nose  "  "mpg"
##  "width of nose   "  "cyl"
##  "width of ear    "  "disp"
##  "height of ear   "  "hp"
```

```r
# USArrests contains four crime-related variables
# for the 50 U.S. states.
head(USArrests)
```

```
##            Murder Assault UrbanPop Rape
## Alabama      13.2     236       58 21.2
## Alaska       10.0     263       48 44.5
## Arizona       8.1     294       80 31.0
## Arkansas      8.8     190       50 19.5
## California    9.0     276       91 40.6
## Colorado      7.9     204       78 38.7
```

```r
par(mar = rep(0, 4))

faces(USArrests, cex = 0.8)
```




```
## effect of variables:
##  modified item       Var
##  "height of face   " "Murder"
##  "width of face    " "Assault"
##  "structure of face" "UrbanPop"
##  "height of mouth  " "Rape"
##  "width of mouth   " "Murder"
##  "smiling          " "Assault"
##  "height of eyes   " "UrbanPop"
##  "width of eyes    " "Rape"
##  "height of hair   " "Murder"
##  "width of hair   "  "Assault"
##  "style of hair   "  "UrbanPop"
##  "height of nose  "  "Rape"
##  "width of nose   "  "Murder"
##  "width of ear    "  "Assault"
##  "height of ear   "  "UrbanPop"
```

The face itself should not be interpreted literally. The purpose is to look for observations that appear similar or unusual across several variables.

A major limitation is that the conclusions can depend strongly on which variables are mapped to which facial features. Some facial characteristics are visually much more prominent than others, so the display may unintentionally emphasize certain variables.

### Visualizing a Correlation Matrix

For a dataset with many quantitative variables, the numerical correlation matrix can become difficult to inspect directly.

A **correlation heatmap** represents each pairwise correlation using color. This makes it easier to identify:

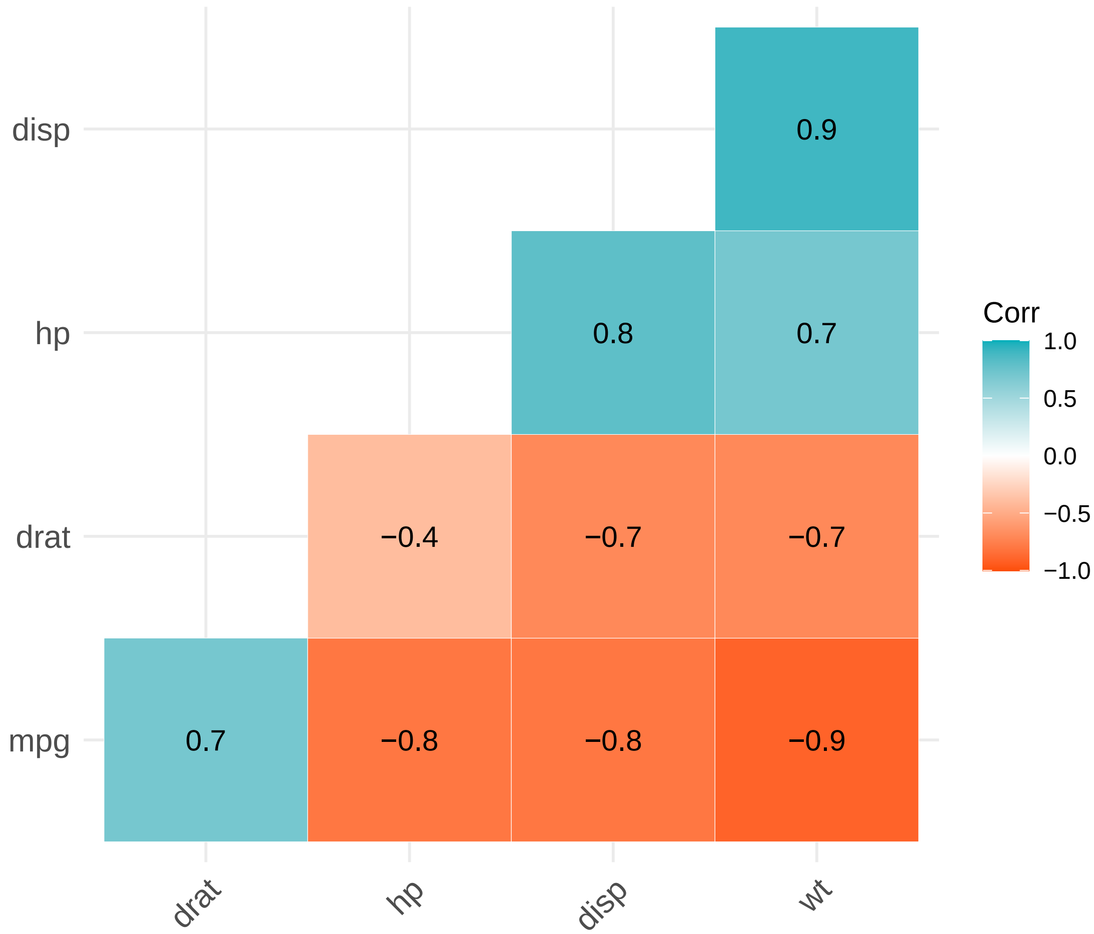
- groups of highly correlated variables;
- positive versus negative associations;
- weak relationships; and
- possible redundancy among variables.

Here we return to the selected `mtcars` variables used previously.

```r
library(ggcorrplot)
```

```
## Warning: package 'ggcorrplot' was built under R version 4.4.3
```

```r
# Compute the sample correlation matrix.
corr <- cor(car)

# Round only for printed numerical labels.
corr_round <- round(corr, 1)

# Calculate p-values associated with tests of zero correlation.
corr_p <- cor_pmat(car)

ggcorrplot(corr_round, p.mat = corr_p,
  hc.order = TRUE, type = "lower",
  color = c("#FC4E07", "white", "#00AFBB"),
  outline.col = "white", lab = TRUE)
```




Several arguments deserve explanation:

`type = "lower"` displays only the lower triangle because a correlation matrix is symmetric.

`hc.order = TRUE` uses hierarchical clustering to reorder the variables so that variables with similar correlation patterns tend to appear together.

`lab = TRUE` prints the numerical correlation coefficients within the colored cells.

The argument `p.mat` provides p-values for tests of zero pairwise correlation. Depending on the plotting defaults, nonsignificant correlations can be treated differently in the display.

The heatmap is not a replacement for understanding the correlation matrix; rather, it makes its structure easier to see.

Strong correlations may indicate that several variables contain overlapping information. This idea will become important later when we study methods such as **principal component analysis**, where correlated variables can often be summarized by a smaller number of underlying dimensions.